

ISI BANGALORE · M.SC. LIBRARY & INFORMATION SCIENCE · SEMESTER III

Information Retrieval, Read as Terrain

A first-principles field guide to Paper 11 — where document image analysis, pattern recognition and information retrieval are treated as one continuous landscape, exactly as Dr. Daya Sagar teaches it.

Course: **Information Retrieval**

Instructor: **Dr. Daya Sagar**

Prepared as: **Primary study material**

Level: **Zero prior assumed**

Your professor moves fast because in his mind, a scanned page, a satellite image, a mountain range and a search query are all the *same kind of object*: a rectangle of numbers with peaks, valleys, and a skeleton hiding inside it. These notes slow that down. We build every idea from nothing — no prior image processing, no prior IR — and every chapter ends by tying the idea back to search engines, OCR, and digital libraries, because that is where this course is ultimately going. Read the chapters in order; each one leans on the last.

TABLE OF CONTENTS

01 Information Retrieval — The Big Picture

1.1 What "Information Retrieval" Means

1.2 Extraction vs. Retrieval

02 Data as Function, Object and Skeleton

2.1 The Three Faces of Data

03 How Space Gets Organised: Packing and Granularity

3.1 Delimiter Space Packing

3.2 Rectangular Granular Space

04 The Geographical Topography Analogy

4.1 An Image Is a Landscape

05 Pixels and the Digital Image

5.1 Pixels

5.2 The Digital Image

5.3 The Function $f(x,y) = z$

5.4 Pixel Intensity

5.5 Why Intensity Runs 0–255

5.6 8-bit Images

06 Binary Images, Thresholds and Energy

6.1 The Binary Image

6.2 Threshold and $f^T(x,y)$

6.3 Image Energy and $\Sigma f(x,y) = A$

07 The Histogram — Turning Terrain into a Graph

7.1 What a Histogram Is

7.2 Percentage of Pixels

7.3 Unimodal, Bimodal, Multimodal Histograms

7.4 Peaks and Valleys

7.5 Choosing a Threshold from the Histogram

08 Synthesis — From Pixels to Search Engines

CHAPTER 01

Information Retrieval — The Big Picture

Before any pixel or formula, you need to know what problem this entire course is solving.

1.1 What "Information Retrieval" Means

DEFINITION

Information Retrieval (IR) is the science and engineering of finding material — usually unstructured material like text, images, or documents — from within a large collection, that satisfies a person's *information need*. It is not about computing an exact answer (like a database query does); it is about finding the most *relevant* items among many, when relevance itself is fuzzy and a matter of degree.

WHY THIS CONCEPT IS NEEDED

A library holds a million books. A search engine indexes trillions of pages. No human can check them one by one. IR exists because collections are too large for exhaustive human search, and because what a person "needs" is rarely expressed as a precise, database-style query — it's expressed as a rough phrase, a blurry photo, or a partial sentence, and the system must still find what's useful.

REAL-LIFE INTUITION

Think of a librarian before computers existed. A visitor says, "I want something about how rivers shape valleys." The librarian doesn't know a book titled exactly that — but she knows the geography section, she recalls a book on erosion, she flips through a card catalogue of subject headings. That entire mental process — narrowing a vague need down to a short, ranked list of promising books — is Information Retrieval. Google, library OPACs, and OCR-searchable PDF archives are just industrial-scale versions of that librarian.

SIMPLE EXPLANATION

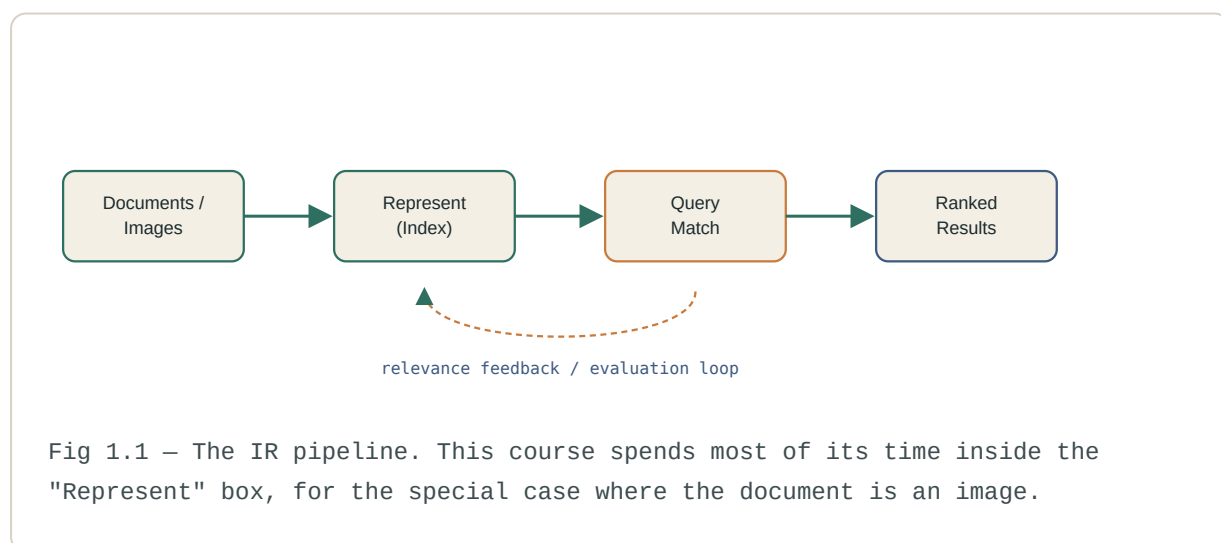
You type a few words into a search box. The system does not "understand" your sentence the way a person does. Instead, it represents both your query and every document in its collection as some mathematical object (a set of words, a vector, a probability), compares them, and returns the documents whose representation is "closest" to your query's representation. IR is the theory of what "closest" should mean and how to compute it fast over huge collections.

TECHNICAL EXPLANATION

An IR system has three standing pieces:

- **A collection** of documents (text, images, audio, video, or mixed — this course will show that an image is just another kind of "document" once you extract the right features from it).
- **An index** — a precomputed structure (e.g., an inverted index of word → list of documents) that lets the system avoid scanning every document for every query.
- **A ranking/matching mechanism** — a model (Boolean, Vector Space, Probabilistic, or neural) that scores how well each document matches a query, so results can be sorted by relevance.

The overall pipeline is always: **acquire** → **represent (index)** → **match (retrieve)** → **rank** → **evaluate**. Everything else in this course — pixels, thresholds, histograms — will turn out to be about step 2: *how do you represent something (in our case, an image) so it can be indexed and matched at all?*



MATHEMATICAL EXPLANATION

IR does not have one single formula the way physics has $F = ma$, but it has one core idea worth writing down now because everything downstream refers back to it: a document D and a query Q are each turned into some representation, and a similarity function $\text{sim}(D, Q)$ produces a real number — a score. Retrieval is simply:

$$\text{Rank}(D_1, D_2, \dots, D_n) = \text{sort by } \text{sim}(D_i, Q), \text{ descending}$$

$D_1 \dots D_n$ — every document in the collection.

Q — the user's query (words, an image, a sketch).

$\text{sim}(D, Q)$ — a similarity score; a bigger number means "more relevant."

Why it exists: without a numeric score, there is no principled way to decide which of a million documents to show first. Every IR model (Boolean, Vector, Probabilistic) is really just a different definition of sim .

WORKED EXAMPLE

EXAMPLE

Suppose a 3-document library has: D_1 = "digital image histogram", D_2 = "library classification systems", D_3 = "image thresholding and histogram". A user queries Q = "image histogram". Using the simplest possible similarity — count of shared words — $\text{sim}(D_1, Q)=2$, $\text{sim}(D_2, Q)=0$, $\text{sim}(D_3, Q)=2$. D_1 and D_3 tie for first place, D_2 is not retrieved at all. This is the entire idea of ranking, just with a tiny, honest vocabulary.

APPLICATIONS

- Web search engines (Google, Bing)
- Library catalogues / OPACs
- Legal and medical document search
- Reverse image search
- Enterprise search over emails and files

CONNECTION TO INFORMATION RETRIEVAL

- **OCR:** before text on a scanned page can be retrieved, it must first be turned into machine-readable text — which needs everything this course covers (binarization, thresholding) before any word-matching happens.
- **Google Search:** represents web pages as huge sparse vectors of words, IR's Vector Space Model in production at planetary scale.
- **Library search:** subject headings and classification numbers are hand-built "representations" of books, an older, human-curated cousin of automatic indexing.
- **Image retrieval:** the "document" is now a picture; "words" become visual features (edges, shapes, histograms) — the subject of the rest of this course.
- **Digital libraries:** combine all of the above — scanned pages, metadata, and full-text search — into one retrieval system.

COMMON INTERVIEW / EXAM QUESTIONS

- Define Information Retrieval and distinguish it from data retrieval (databases).
- Draw and explain the general IR system architecture.
- Why is relevance considered a "fuzzy" rather than exact match in IR?

Memory trick — "A-R-M-R": Acquire, Represent, Match, Rank. Say it like a workout: you always ARM your retrieval system before it can fire.

KEY POINTS TO REMEMBER

- IR is not exact database lookup; it deals with fuzzy relevance over unstructured collections.
- Every IR system needs: a collection, an index (representation), and a matching/ranking function.
- Everything about images in this course is really about Step 2: *representation*.

1.2 Extraction vs. Retrieval

DEFINITION

Extraction is the act of pulling a specific piece of structure or content *out of* a single object — e.g., pulling the text out of one scanned page, or pulling an object's outline out of one image. **Retrieval** is the act of finding the right object(s) *from among many*, based on a need. Extraction happens on one item; retrieval happens across a collection.

WHY THIS CONCEPT IS NEEDED

Students often conflate the two because in practice they happen back-to-back: you extract features from every image in a collection, then retrieve the best-matching image for a query. But they solve different problems, and this course's mathematics belongs almost entirely to *extraction* — extracting objects, skeletons, thresholds and histograms from an image — while classical IR theory (Boolean/Vector/Probabilistic models) belongs to *retrieval*.

REAL-LIFE INTUITION

Extraction is like a chef removing the bones from one fish — a precise, local operation on a single item. Retrieval is like a customer at a fish market pointing at the best fish among a hundred on ice — a comparative, collection-wide decision. You need boned fish (extraction) before you can meaningfully compare and pick (retrieval), but boning a fish and picking a fish are different skills.

SIMPLE EXPLANATION

If someone hands you one photograph and asks "outline the car in this picture," that's extraction. If someone hands you ten thousand photographs and asks "give me every picture that contains a car," that's retrieval.

TECHNICAL EXPLANATION

Extraction operates at the level of a single document/image: segmentation, feature extraction, object/skeleton extraction, OCR character extraction. Its output is usually a *compact representation* — a feature vector, a set of coordinates, a binary mask. Retrieval operates at the level of the whole collection: it takes those compact

representations (already extracted) and performs indexing, similarity scoring, and ranking. In a modern pipeline, extraction is almost always the pre-processing stage that feeds retrieval.

	Extraction	Retrieval
Scope	One document / one image	Entire collection
Goal	Pull out structure, features, or content	Find the most relevant items
Typical output	Feature vector, mask, text, skeleton	Ranked list of documents
Example operation	Thresholding an image to find text regions	Matching a query against an index of many documents
Course topic it maps to	Pixels, thresholds, histograms, skeletons	Boolean/Vector/Probabilistic IR models

MATHEMATICAL EXPLANATION

Extraction can be written as a function operating on one image:

$$\text{Extract: } \mathbf{I} \rightarrow \mathbf{F}$$

I – one input image (or document). **F** – a feature representation (e.g. a histogram, a skeleton, a set of keypoints). Only one input, one output – a local operation.

Retrieval, in contrast, operates on the whole collection at once:

Retrieve: $\{F_1, F_2, \dots, F_n\}, Q \rightarrow \text{ranked subset}$

$F_1 \dots F_n$ – the extracted features of every document in the collection. Q – the query's own feature representation. The output is a subset, ranked – many inputs, one ordered output – a global, comparative operation.

WORKED EXAMPLE

EXAMPLE

A library digitises 500 old manuscripts. **Extraction:** for each manuscript image, compute a histogram and threshold it to separate ink from paper, then extract the skeleton of each character stroke. This is done 500 times, once per manuscript, independently. **Retrieval:** a researcher later searches "show me manuscripts with dense cursive handwriting" — the system compares the 500 stored feature sets against this need and ranks the manuscripts. Extraction happened once per document; retrieval happens once per query, across all 500 at once.

CONNECTION TO INFORMATION RETRIEVAL

- **OCR:** character/word extraction from a scanned page is extraction; searching across a digitised newspaper archive for a keyword is retrieval.
- **Image Retrieval:** extracting a colour histogram from one photo is extraction; finding all photos with a similar histogram is retrieval.
- **Digital Libraries:** extracting metadata/text from a PDF is extraction; a patron's catalogue search is retrieval.

COMMON INTERVIEW / EXAM QUESTIONS

- Differentiate extraction and retrieval with one example each from document image analysis.
- Why must extraction usually precede retrieval in a multimedia IR pipeline?

Memory trick — "Extraction is 1-to-1, Retrieval is many-to-1": one image goes in, one feature set comes out (extraction); many feature sets go in, one ranked list comes out (retrieval).

KEY POINTS TO REMEMBER

- Extraction = local, per-document. Retrieval = global, per-collection.
- Extraction produces the representation that retrieval later compares.
- Most of this course's mathematics (pixels to histograms) is extraction; classic IR models handle retrieval.

[↑ Back to Table of Contents](#)

CHAPTER 02

Data as Function, Object and Skeleton

Dr. Sagar's central organising idea: any piece of data can be looked at three ways — what it computes, what it contains, and what holds it together.

2.1 The Three Faces of Data

DEFINITION

Any rectangular block of data (most importantly, a digital image) can be understood at three distinct levels:

- **Function** — the data viewed as a mathematical function that assigns a value to every position (e.g. $f(x,y) = z$, giving a brightness value to every pixel location).
- **Object** — the data viewed as containing meaningful "things" (a letter, a face, a region of dense vegetation) that are distinguishable from the background.
- **Skeleton** — the data viewed as a thin, simplified core structure that preserves the shape/topology of an object while discarding its bulk (e.g. reducing a thick handwritten stroke to a one-pixel-wide centreline).

WHY THIS CONCEPT IS NEEDED

Different tasks in document image analysis, pattern recognition and IR need different "views" of the same data. If you want to compute brightness statistics, you need the *function* view. If you want to know "is there a car in this photo," you need the *object* view. If you want to recognise a handwritten character regardless of pen thickness, you need the *skeleton* view. Confusing these three levels is the single biggest source of confusion when a professor jumps between image processing, pattern recognition and mathematical morphology in one breath — he is simply switching which of the three views he is discussing.

REAL-LIFE INTUITION

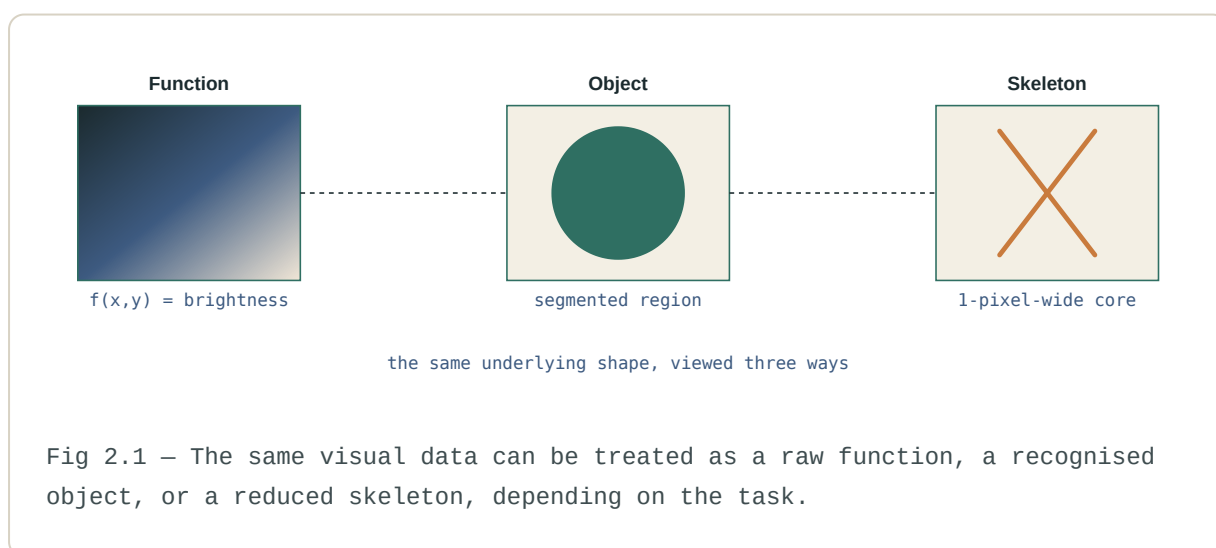
Think of a human hand. As a **function**, it is a continuous surface of skin with varying height at every point (like a topographic map of the palm). As an **object**, it is "a hand" — a single recognisable unit, distinct from the arm, the table, the background. As a **skeleton**, it is the stick-figure your brain draws when it thinks of "hand shape" — one line per finger, meeting at a palm point, with all thickness removed. Three views, one hand.

SIMPLE EXPLANATION

Function = "what number lives at each spot." Object = "what thing is at this spot." Skeleton = "what is the thinnest possible sketch that still shows the shape." You will use all three constantly: the function view to do maths on the image, the object view to recognise or segment it, and the skeleton view to compress a shape down to its essential structure for matching or recognition.

TECHNICAL EXPLANATION

In document image analysis, a scanned page is first captured as a *function* $f(x,y)$ (raw pixel intensities — Chapter 5). Thresholding and segmentation convert regions of that function into *objects* (connected components — candidate characters, words, or images on the page). Skeletonisation (a mathematical morphology operation) then thins each object to a *skeleton* — a topology-preserving, one-pixel-wide representation used for stroke analysis, character recognition, and matching, because it removes irrelevant thickness/stroke-width variation while keeping the shape that actually distinguishes one letter from another.



MATHEMATICAL EXPLANATION

$$\text{Data} \rightarrow \{ f(x,y) , \mathbf{0} = \{0_1, \dots, 0_k\} , S = \text{skeleton}(\mathbf{0}) \}$$

f(x,y) – the function view: a value at every coordinate (see Chapter 5).

0 = {0₁, ..., 0_k} – the object view: the image broken into k distinguishable regions/objects.

S = skeleton(0) – a thinning operator applied to an object, producing its topological "backbone."

Why it exists: writing data this way makes explicit that "the image" isn't one fixed thing – it is whichever representation is useful for the operation you are about to perform.

WORKED EXAMPLE

EXAMPLE

A scanned signature. As a **function**, it's a grid of grey values. After thresholding, the ink pixels form one connected **object** against the white background. Skeletonising that object collapses every thick pen-stroke to a single-pixel-wide curve — the **skeleton** — which is what a signature-verification system actually compares, because it cares about stroke *shape*, not how thick the pen happened to be.

View	Answers the question	Used for
Function	"What value is here?"	Filtering, enhancement, statistics, histograms
Object	"What thing is here?"	Segmentation, detection, classification
Skeleton	"What is this thing's essential shape?"	Character/shape recognition, matching, morphology

CONNECTION TO INFORMATION RETRIEVAL

- **OCR:** function → threshold → object (a candidate character) → skeleton (used for stroke-based recognition) → recognised text → now retrievable by keyword search.
- **Pattern Recognition:** skeletons are classic shape descriptors used to classify objects into categories.
- **Document Image Analysis:** the whole field is essentially the pipeline of moving data through these three views in order.
- **Multimedia Retrieval:** object-level and skeleton-level features are exactly the kind of compact representations that get indexed for image/video search.

COMMON INTERVIEW / EXAM QUESTIONS

- Explain the function–object–skeleton hierarchy with a document image example.
- Why is skeletonisation useful for character recognition but not for colour analysis?

Memory trick — "F-O-S, like Flesh, Outline, Skeleton": every image has flesh (values), an outline (objects), and a skeleton (thinned structure) — just like a body.

KEY POINTS TO REMEMBER

- Function, object and skeleton are three complementary views of the same underlying data, not three different datasets.
- Choice of view depends on the task: measurement (function), detection (object), shape matching (skeleton).
- Document image pipelines typically move Function → Object → Skeleton in that order.

[↑ Back to Table of Contents](#)

CHAPTER 03

How Space Gets Organised: Packing and Granularity

Before we can talk about pixels, we need to understand how continuous space is cut into discrete pieces in the first place.

3.1 Delimiter Space Packing

DEFINITION

Delimiter Space Packing refers to the idea of dividing a continuous space (a page, a photograph, a geographic area) into discrete, non-overlapping units by inserting **delimiters** — boundary markers that say "this unit ends here, the next one starts." "Packing" means these units are laid out to completely fill the space with no gaps and no overlaps.

WHY THIS CONCEPT IS NEEDED

A digital device cannot store or process a truly continuous signal (infinite precision, infinite points). It must break continuous space into a finite number of manageable, addressable chunks. Before we can even define "a pixel," we first need the more general idea that any continuous space can be *packed* using *delimiters* into countable, indexable units — this is the conceptual ancestor of the pixel grid.

REAL-LIFE INTUITION

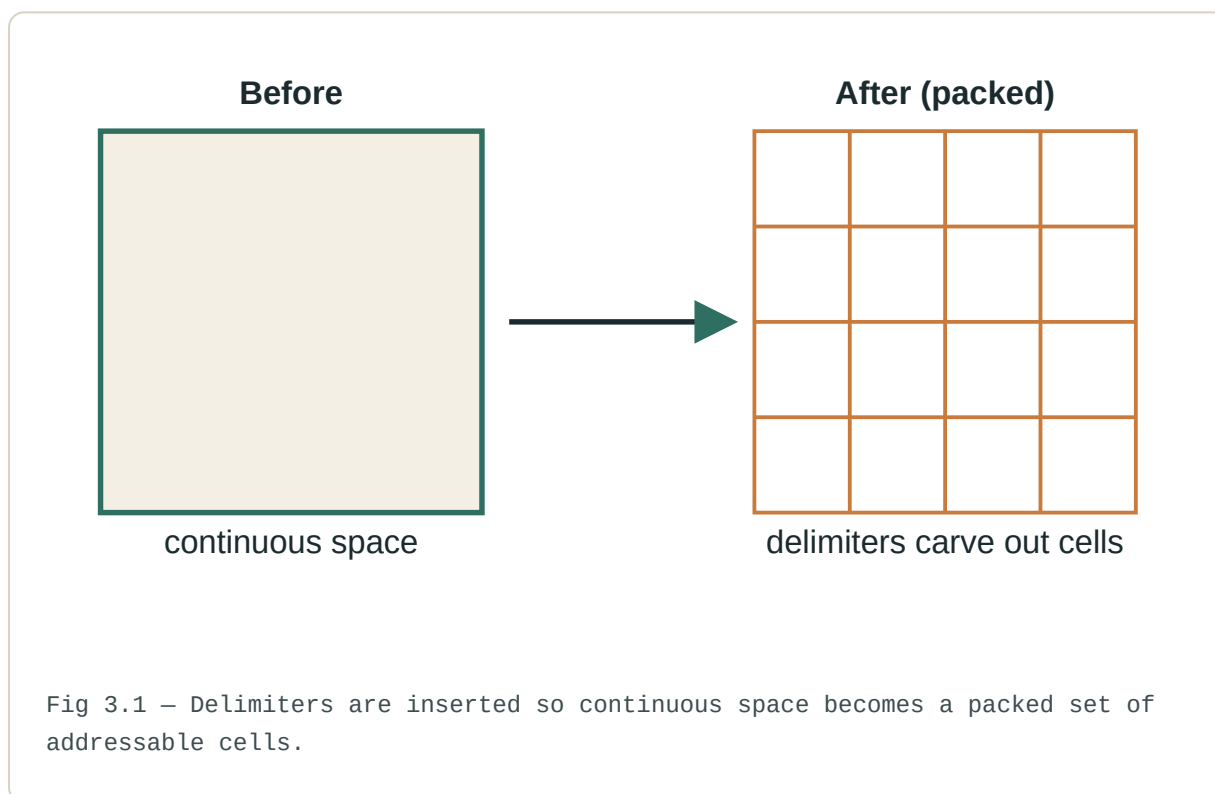
Think of a chocolate bar. It's a single continuous rectangle, but it has grooves — delimiters — pressed into it that mark where each small square begins and ends. You can now refer to "the third square from the left" even though, physically, the chocolate is one connected piece. The grooves don't change the chocolate; they change how you can *address* it. Delimiter space packing does the same thing to an image plane or a page: it lays an invisible grid of "grooves" so every location can be named and processed individually.

SIMPLE EXPLANATION

Take a blank rectangle. Draw lines across it, edge to edge, both horizontally and vertically, with no gaps between the lines and no overlaps. Each little box formed by the lines is one "packed" unit; the lines themselves are the delimiters. This is exactly what happens, conceptually, right before a scanner or camera turns light falling on a page into a grid of numbers.

TECHNICAL EXPLANATION

Formally, delimiter space packing takes a continuous 2-D region R and partitions it into a set of cells $\{C_1, C_2, \dots, C_n\}$ such that: (a) the union of all cells equals R (complete coverage — packing), and (b) no two cells share interior area (non-overlap — enforced by the delimiters). This is the discretisation step that must happen before "space" can be represented as data at all. It applies not only to images but to any domain that needs to chop continuous extent into discrete, addressable records — this is why the same idea resurfaces in spatial/geographic data models, not just in pixel grids.



MATHEMATICAL EXPLANATION

$$R = C_1 \cup C_2 \cup \dots \cup C_n, \quad C_i \cap C_j = \emptyset \text{ for } i \neq j$$

R – the original continuous region (a page, a scene).

$C_1 \dots C_n$ – the packed cells produced by inserting delimiters.

U (union) – every point of R is covered by some cell (nothing is left undefined).

$\cap = \emptyset$ (empty intersection) – no two cells overlap; every point belongs to exactly one cell.

Why it exists: this pair of conditions is precisely what "packing" means in mathematics – full coverage with zero overlap – and it is the property that lets us later index cells with simple integer coordinates (Chapter 5).

WORKED EXAMPLE

EXAMPLE

A 3 cm × 3 cm patch of a photograph is packed using delimiters spaced 1 cm apart in both directions. This produces 3 × 3 = 9 cells, each addressable as (row, column). No point in the original patch is left out, and no two cells share area — exactly the packing condition above. This 3×3 grid of cells is a miniature version of what becomes a pixel grid once each cell is assigned one intensity value.

CONNECTION TO INFORMATION RETRIEVAL

- **Document Image Analysis:** a scanned page must be packed into a pixel grid before any indexing/OCR can occur.
- **Digital Libraries:** tiling large images (e.g. manuscripts) into packed cells enables efficient storage and progressive loading during retrieval.
- **Multimedia Retrieval:** spatial indexing structures (grids, quad-trees) used to retrieve regions of large images rely on this same packing idea.

COMMON INTERVIEW / EXAM QUESTIONS

- What two conditions must a "packing" of space satisfy?
- Why must continuous image space be discretised before computational processing?

Memory trick — "Pack like tiles, not like puddles": tiles cover a floor with no gaps and no overlaps; puddles overlap and leave gaps. A proper packing behaves like tiles.

KEY POINTS TO REMEMBER

- Delimiters are the boundary markers; packing is the resulting gap-free, overlap-free tiling.
- This is the conceptual step before pixels: turning continuous space into countable cells.

3.2 Rectangular Granular Space

DEFINITION

A **Rectangular Granular Space** is the specific (and by far most common) case of delimiter space packing where every cell (every "granule") is a rectangle — usually a square — and all granules are arranged in uniform rows and columns. "Granular" emphasises that the space is treated as made of small, uniform grains rather than as one smooth continuum.

WHY THIS CONCEPT IS NEEDED

Packing (Section 3.1) tells us space *can* be divided into cells; it doesn't say what shape those cells should be. Rectangles are chosen almost universally in digital imaging because they tile a plane perfectly with no gaps, they align naturally with row/column memory storage in a computer, and they make coordinate arithmetic (addition, distance, indexing) trivially simple compared to, say, hexagonal or irregular cells.

REAL-LIFE INTUITION

A rectangular granular space is like a chess board or a spreadsheet: every cell is the same rectangular shape, arranged in tidy rows and columns, so "cell (3, 5)" is unambiguous and easy to compute. Compare this to a jigsaw puzzle, where pieces are irregular and there's no simple formula for "the piece two positions to the right" — a rectangular grid avoids exactly that problem.

SIMPLE EXPLANATION

Every digital image you have ever seen is a rectangular granular space: a grid of tiny square cells (pixels), each holding one value, arranged in evenly spaced rows and columns.

TECHNICAL EXPLANATION

A rectangular granular space is defined by a fixed cell width and height, and two integer indices per cell: a row index and a column index. Because every granule is congruent (identical in shape and size) and axis aligned, this structure supports constant-time addressing: given (row, column), the cell's location can be computed directly by multiplication, with no search required. This is exactly the coordinate system used for every digital image, satellite raster, and scanned document page.

Rectangular Granular Space

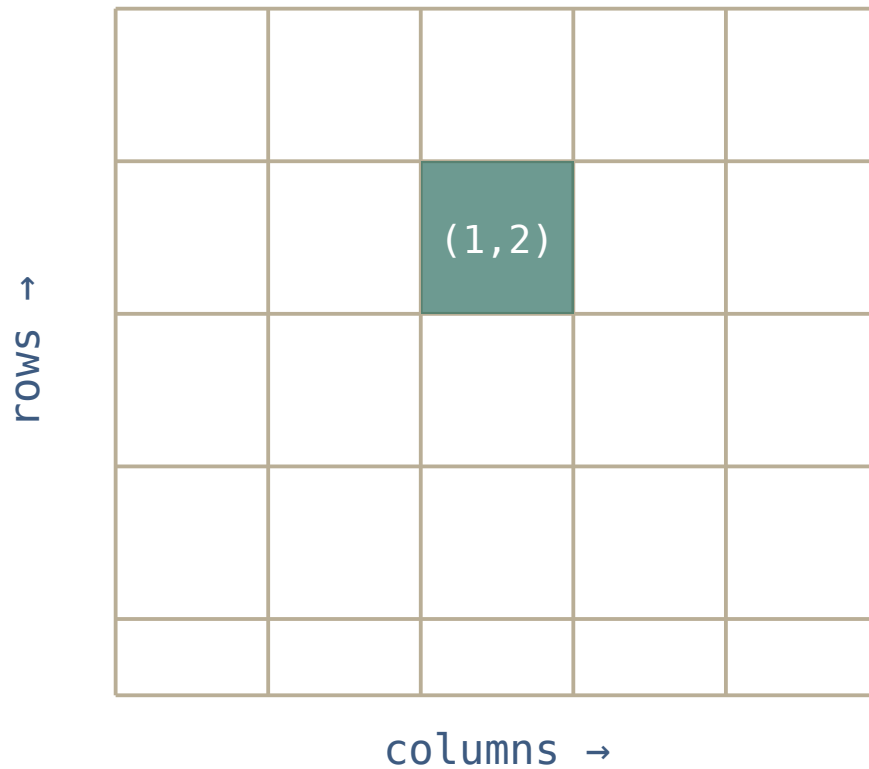


Fig 3.2 – Every granule is an identical rectangle; any cell is addressed simply as (row, column).

MATHEMATICAL EXPLANATION

$$\text{Cell}(r, c) = [r \cdot w, (r+1) \cdot w] \times [c \cdot h, (c+1) \cdot h]$$

r, c – the row and column index of a granule (whole numbers: 0, 1, 2, ...).

w, h – the fixed width and height of every granule.

[r·w, (r+1)·w] – the horizontal extent of granule (r,c); every granule has exactly the same extent, just shifted.

Why it exists: this formula shows that once you know (r, c), the granule's exact location follows from simple multiplication – no lookup table, no search. That efficiency is the entire reason rectangular grids dominate digital imaging.

WORKED EXAMPLE

EXAMPLE

A granule width/height of $w = h = 1$ unit. Granule ($r=2, c=3$) occupies the rectangle $[2,3] \times [3,4]$. To find granule (5, 0), you don't need to search anything — you simply compute $[5,6] \times [0,1]$. This constant-time addressing is exactly what lets a 10-million-pixel photograph be accessed instantly at any location.

Property	Delimiter Space Packing (general)	Rectangular Granular Space (special case)
Cell shape	Any shape, as long as it tiles	Always rectangle/square
Addressing	May need custom indexing	Simple (row, column) arithmetic
Used for	General spatial partitioning theory	Every practical digital image / raster

CONNECTION TO INFORMATION RETRIEVAL

- **Digital Libraries / Image Retrieval:** raster images stored as rectangular grids allow region-based indexing (search by image tile) and fast random access needed for retrieval systems.
- **OCR:** pixel-grid rectangular addressing is the substrate on which every later text-extraction operation runs.
- **Pattern Recognition:** features are almost always computed over rectangular sub-windows of an image precisely because the addressing is so simple.

COMMON INTERVIEW / EXAM QUESTIONS

- Why are rectangular (rather than hexagonal or irregular) cells used almost universally in digital imaging?
- Give the formula for locating a granule given its row and column index.

Memory trick — "Granular = Grid-like": both words start with "gr" — a granular space is a grid of identical little rectangles.

KEY POINTS TO REMEMBER

- Rectangular granular space is delimiter space packing specialised to identical rectangular cells in rows/columns.
- It enables constant-time, formula-based addressing — the foundation for the pixel grid in Chapter 5.

[↑ Back to Table of Contents](#)

CHAPTER 04

The Geographical Topography Analogy

The single mental model that will make every later chapter — thresholds, histograms, peaks, valleys — click into place instantly.

4.1 An Image Is a Landscape

DEFINITION

The **Geographical Topography Analogy** treats a grayscale image exactly like a physical terrain map: each pixel's (x, y) location is like a point on a map, and its intensity value is like the *elevation* at that point. Bright pixels are "high ground" (mountains/peaks); dark pixels are "low ground" (valleys/basins). An entire image becomes a 3-D landscape once you plot intensity as height.

WHY THIS CONCEPT IS NEEDED

Human intuition for terrain — peaks, valleys, ridgelines, sea level — is extremely well developed (everyone understands mountains and rivers) but intuition for "arrays of grey numbers" is not. By mapping every image concept onto a landscape concept, abstract operations (thresholding, histogram peaks, mathematical morphology) become physically visualisable rather than purely symbolic. This is precisely why Dr. Sagar — whose research area explicitly includes mathematical morphology and geographic topography — introduces it early: it is the single bridge connecting document image analysis to geographic information science.

REAL-LIFE INTUITION

Imagine flying over a mountain range at night with a heat-sensitive camera that shows height as brightness: the tallest peaks glow brightest, the deepest valleys stay darkest. That glowing image *is* a grayscale image, and the mountain range beneath it *is* its topography. Reading a grayscale photo of anything — a face, a page of text, a satellite photo — the same way you'd read that glowing mountain photo is the whole trick.

SIMPLE EXPLANATION

Wherever the image is bright, imagine a hill. Wherever it's dark, imagine a valley. A histogram (Chapter 7) is just a way of counting "how much land exists at each elevation" without needing to look at the whole map — exactly like a geographer summarising a mountain range by how much area lies at each altitude band.

TECHNICAL EXPLANATION

Formally, we lift a 2-D image $f(x,y)$ into a 3-D surface by treating intensity as height: the surface is the set of points $(x, y, f(x,y))$. Ridges of this surface correspond to bright, connected structures (e.g., a bright object's edge); basins correspond to dark, connected regions (e.g., ink on a page, or shadowed regions). Segmenting an image into "object" and "background" (Chapter 6, thresholding) becomes equivalent to choosing a "sea level" and asking which land is above it (foreground) and which is submerged (background) — exactly the geomorphological operation of flooding a terrain to a chosen elevation.

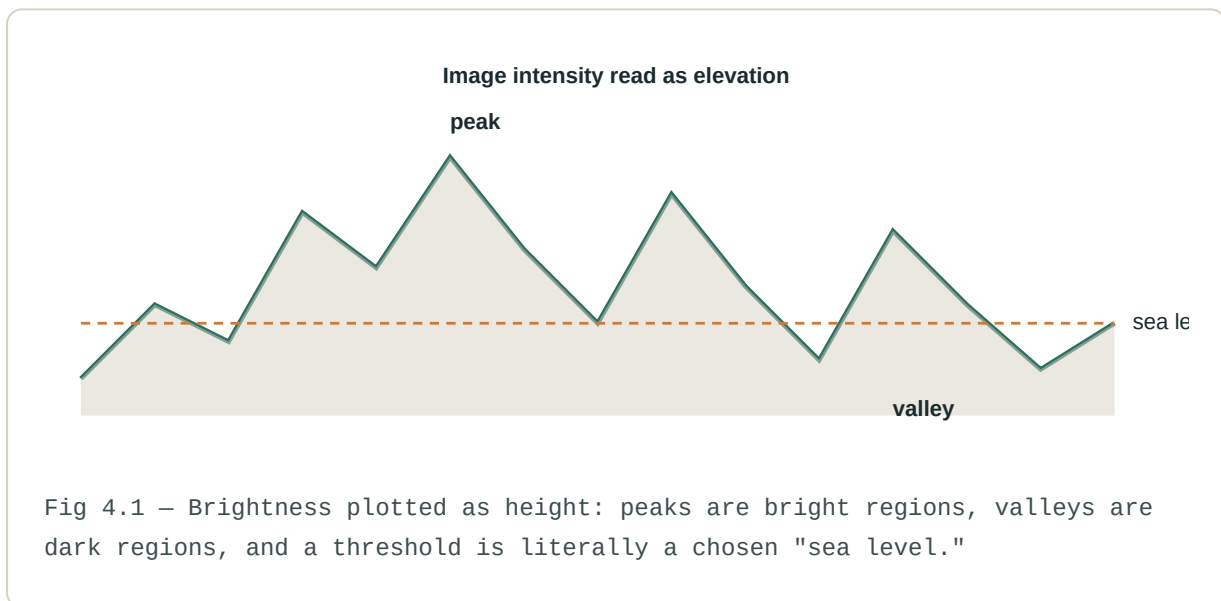


Fig 4.1 – Brightness plotted as height: peaks are bright regions, valleys are dark regions, and a threshold is literally a chosen "sea level."

MATHEMATICAL EXPLANATION

$$\text{Surface} = \{(x, y, z) : z = f(x, y)\}$$

x, y – horizontal position on the map/image (Chapter 5).

z = f(x,y) – elevation at that position, which for an image is simply pixel intensity.

Why it exists: writing the image this way makes it literally a terrain surface, so every geographic concept (contour lines, peaks, watershed) has a direct, ready-made translation into image processing.

WORKED EXAMPLE

EXAMPLE

A scanned page of dark text on a white background: white paper = high elevation (bright = high intensity), black ink = low elevation. Choosing a threshold to separate text from paper (Chapter 6) is the same act as a geographer choosing a "flood level" that submerges the ink valleys while leaving the paper peaks dry — the flooded (submerged) area becomes the "text" mask.

CONNECTION TO INFORMATION RETRIEVAL

- **Document Image Analysis:** the topography view underlies watershed segmentation, a classic method for separating touching characters or objects.
- **Geographic Information Science:** the same mathematical-morphology machinery (used in ISI's own GIS elective) analyses real terrain, satellite imagery, and digital elevation models.
- **Image Retrieval:** "peaks" and "valleys" of an image's histogram (not the pixel surface itself) are later used as compact features for indexing and comparing images.

COMMON INTERVIEW / EXAM QUESTIONS

- Explain how a grayscale image can be interpreted as a topographic surface.
- How does the topography analogy help explain image thresholding?

Memory trick — "Bright = high, dark = low, threshold = sea level." If you remember only this one sentence, the rest of the course's vocabulary (peaks, valleys, watershed) falls into place automatically.

KEY POINTS TO REMEMBER

- Intensity is treated as elevation; the 2-D image becomes a 3-D landscape.
- Thresholding = choosing a sea level; histogram peaks/valleys = amount of "land" at each elevation.
- This analogy is the reason mathematical morphology, GIS and document image analysis share so much mathematics.

[↑ Back to Table of Contents](#)

CHAPTER 05

Pixels and the Digital Image

Now we zoom into a single granule of our rectangular grid and ask: what number actually lives inside it, and why?

5.1 Pixels

DEFINITION

A **pixel** ("picture element") is the smallest addressable unit of a digital image — one granule of the rectangular granular space from Chapter 3 — holding a single value that represents brightness (grayscale) or colour at that location.

WHY THIS CONCEPT IS NEEDED

A computer cannot store a continuous image (it would need infinite numbers for infinite points). The pixel is the practical compromise: divide the image into finitely many cells and store one representative number per cell. Every later operation — thresholding, histograms, filtering — operates on this grid of pixels, never on the original continuous scene.

REAL-LIFE INTUITION

Think of a mosaic made of small coloured tiles. From far away it looks like a smooth picture, but up close it's just a grid of individual tiles, each one flat, uniform colour. A digital image is a mosaic where the "tiles" are pixels — each one a tiny, uniform square holding exactly one number.

SIMPLE EXPLANATION

Zoom into any digital photo far enough and it turns blocky — those blocks are pixels. Each block cannot be subdivided further; it is the atomic unit of the image.

TECHNICAL EXPLANATION

A digital image of size $M \times N$ contains $M \times N$ pixels, arranged in M rows and N columns (Chapter 3's rectangular granular space with granule count fixed). Each pixel is indexed by an integer coordinate pair (x, y) and stores one intensity value (grayscale) or a triplet of values (colour, e.g. Red/Green/Blue).

A 5×5 Pixel Grid (values = grey levels)

10	60	110	150	230
40	90	130	180	250
20	75	120	170	240

← row

Each cell = one pixel = one (x, y) location = one intensity value
column →

Fig 5.1 – A tiny 5×5 image. Darker cells hold lower numbers (closer to 0); lighter cells hold higher numbers (closer to 255).

TERMINOLOGY CHECK

Synonyms you may see for "pixel": picture element, image element, pel (older texts). All mean the same thing.

APPLICATIONS

- Every digital photo, scan, X-ray, or satellite image is fundamentally a grid of pixels.

CONNECTION TO INFORMATION RETRIEVAL

- **Image Retrieval:** pixels are the raw material from which every higher-level feature (histogram, edge, shape) used for search is eventually computed.
- **OCR:** characters are recognised only after being reconstructed from patterns of pixels.

5.2 The Digital Image

DEFINITION

A **digital image** is a finite, discrete, two-dimensional array of pixels — the complete rectangular granular space (Chapter 3), fully populated with intensity values.

WHY THIS CONCEPT IS NEEDED

"Pixel" describes one cell; "digital image" describes the whole array. We need the aggregate concept because almost every operation in this course — histograms, thresholding, energy — is defined over the entire image, not over a single pixel in isolation.

REAL-LIFE INTUITION

If a pixel is one tile in a mosaic, a digital image is the entire finished mosaic — the complete picture made of all the tiles laid together.

TECHNICAL EXPLANATION

A digital image is formed by two processes applied to a continuous scene: **sampling** (deciding *where* to measure — this fixes the $M \times N$ grid, i.e. Chapter 3's packing) and **quantisation** (deciding *what values* are allowed at each sample — this fixes the 0–255 range discussed in Section 5.5). Together, sampling + quantisation turn a continuous scene into a finite grid of finite-precision numbers: a digital image.

Step

Question it answers

What it fixes

Sampling	Where do we measure?	Image resolution ($M \times N$, rows $\times$ columns)
Quantisation	What values can a measurement take?	Intensity depth (e.g. 0–255, Section 5.6)

CONNECTION TO INFORMATION RETRIEVAL

- **Digital Libraries:** digitisation of manuscripts is exactly this sampling + quantisation process, producing the digital image objects that libraries then catalogue and make retrievable.

5.3 The Function $f(x,y) = z$

DEFINITION

A digital image is formally written as a function $f(x, y) = z$, meaning: at horizontal position x and vertical position y , the image has intensity value z .

WHY THIS CONCEPT IS NEEDED

Writing an image as a mathematical function (rather than just "a picture") is what allows us to apply the entire toolbox of mathematics — sums, derivatives, transforms, statistics — to images. Every later formula in this course (energy, histograms) is built directly on top of $f(x,y)$.

SIMPLE EXPLANATION

Think of f as a lookup rule: "tell me a location, I'll tell you the brightness there." That's all a function is — a rule that takes inputs and produces one output.

MATHEMATICAL EXPLANATION

$$f(x, y) = z$$

x – the column position (horizontal coordinate) of a pixel, an integer from 0 to N-1.

y – the row position (vertical coordinate) of a pixel, an integer from 0 to M-1.

f – the "image function," the rule that assigns a value to every (x, y) pair. It is called a *function* because – exactly like $f(x) = x^2$ in ordinary algebra – every valid input (x,y) maps to exactly one output; no location has two different brightness values at once.

z – the output: the intensity (brightness) at location (x,y). For an 8-bit grayscale image, z is an integer from 0 to 255 (Section 5.5-5.6).

Why the notation exists: it lets us treat "the whole image" as a single mathematical object f, instead of talking about millions of separate numbers individually – exactly as $f(x)$ lets algebra talk about a whole curve instead of listing every point on it.

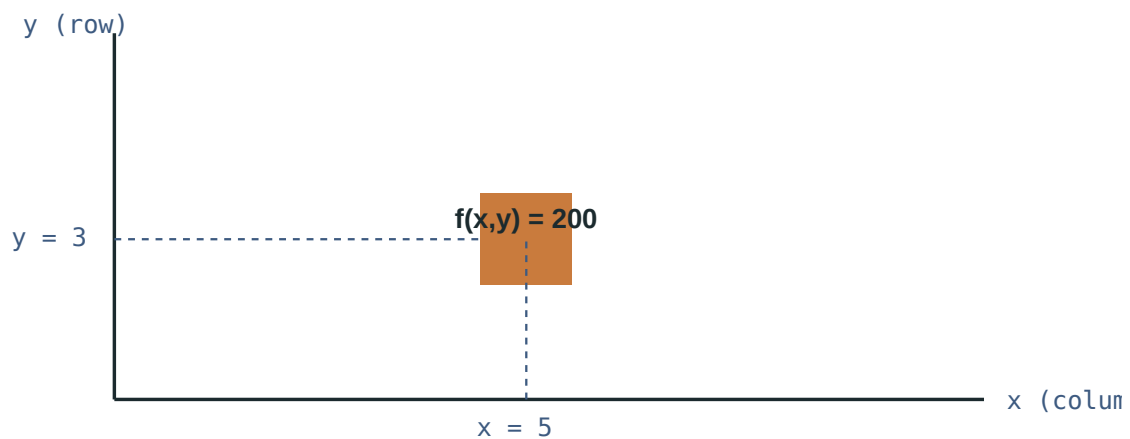


Fig 5.2 – $f(5,3) = 200$ means: "at column 5, row 3, the brightness is 200."

WORKED EXAMPLE

EXAMPLE

Given $f(2,1) = 45$, $f(2,2) = 210$, $f(3,1) = 45$: two different locations, (2,1) and (3,1), happen to share the same brightness (45) — that's allowed; a function

only forbids one input having two outputs, it never forbids two inputs sharing one output.

CONNECTION TO INFORMATION RETRIEVAL

- **Multimedia Retrieval:** every image-based feature used for search (histograms, energy, edges) is computed as some aggregate function of $f(x,y)$ — this notation is the universal starting point.

COMMON INTERVIEW / EXAM QUESTIONS

- Why is a digital image represented as a mathematical function rather than as a mere "picture"?

Memory trick — "x across, y down, f finds, z is found": read $f(x,y)=z$ left to right as a sentence.

5.4 Pixel Intensity

DEFINITION

Pixel intensity (also called grey level or grey value) is the numeric value $z = f(x,y)$ stored at a pixel, representing how bright or dark that point is.

WHY THIS CONCEPT IS NEEDED

Intensity is the actual payload of an image — everything else (coordinates, grid structure) is just "addressing." Without intensity values, the grid would be empty; intensity is what makes an image visually meaningful.

REAL-LIFE INTUITION

Think of a dimmer switch on a light bulb: turning it fully down gives darkness (low intensity, near 0); turning it fully up gives maximum brightness (high intensity, near 255). Every pixel has its own invisible "dimmer switch" set to some value in that range.

TECHNICAL EXPLANATION

In grayscale imaging, intensity is a single number per pixel. In colour imaging, intensity is usually split into three channels (Red, Green, Blue), each with its own 0–255 range, combined to produce the perceived colour. This course, following the professor's treatment, focuses on the single-channel grayscale case, which is also what underlies binary thresholding (Chapter 6).

CONNECTION TO INFORMATION RETRIEVAL

- **Image Retrieval:** intensity distributions (histograms, Chapter 7) are among the oldest and simplest global features used to compare and retrieve images.

Memory trick — "Intensity is the answer, coordinates are the question": (x,y) asks "where," z answers "how bright."

5.5 Why Intensity Runs 0–255

DEFINITION

In the most common digital imaging standard, intensity z is restricted to whole numbers from **0 to 255** inclusive — 256 possible values — because each pixel's intensity is stored using exactly **8 bits** of computer memory (Section 5.6).

WHY THIS CONCEPT IS NEEDED

Students often think 0–255 is an arbitrary convention; it is actually a direct mathematical consequence of how binary computer memory works, and understanding **why** removes a huge amount of confusion when later reading about "8-bit images," "bit depth," or "256 gray levels" in any textbook or paper.

REAL-LIFE INTUITION

Imagine a row of 8 light switches, each either OFF (0) or ON (1). How many distinct on/off patterns can 8 switches make? Each switch doubles the number of possible

WORKED EXAMPLE

EXAMPLE

With 1 bit per pixel: $2^1 = 2$ levels — only pure black (0) or pure white (1); this is exactly a binary image (Chapter 6). With 2 bits per pixel: $2^2 = 4$ levels (0,1,2,3). With 8 bits: $2^8 = 256$ levels (0...255) — the everyday grayscale standard. Notice the pattern: each extra bit doubles the number of available shades.

Bits per pixel	Formula	Number of grey levels	Example use
1	2^1	2	Binary (black/white) image
4	2^4	16	Old low-colour displays
8	2^8	256	Standard grayscale image
16	2^{16}	65,536	Medical/scientific high-precision imaging

CONNECTION TO INFORMATION RETRIEVAL

- **Image Retrieval:** the number of grey levels directly determines histogram resolution (Chapter 7) — a coarser bit depth means a coarser, less discriminative feature for retrieval.
- **OCR:** 8-bit grayscale is typically the working depth before thresholding down to 1-bit binary for character segmentation.

COMMON INTERVIEW / EXAM QUESTIONS

- Derive why an 8-bit image has exactly 256 possible intensity values.
- How many grey levels are possible with 4 bits per pixel? With 10 bits?

Memory trick — "n switches, 2^n patterns": every extra bit doubles your palette.

5.6 8-bit Images

DEFINITION

An **8-bit image** is a digital image in which every pixel's intensity is stored using 8 bits (1 byte), giving 256 possible grey levels (0–255), as derived in Section 5.5.

WHY THIS CONCEPT IS NEEDED

8-bit depth is the de facto standard for grayscale images (and for each colour channel in 24-bit colour images) because 1 byte = 8 bits is the most natural, efficient unit of computer memory, and 256 shades is more than enough for human vision to perceive as smooth, continuous-looking tone.

REAL-LIFE INTUITION

Photograph a black-and-white gradient print with a scanner set to 8-bit depth: the human eye typically cannot distinguish more than about 200–300 shades of grey anyway, so 256 levels look perfectly smooth — going to 16-bit depth mostly matters for later mathematical processing (e.g. medical scans), not for direct viewing.

TECHNICAL EXPLANATION

Storage cost: an 8-bit image of size $M \times N$ requires $M \times N$ bytes for its raw pixel data (before compression). This is also why an 8-bit binary image (Chapter 6) — which really only needs 1 bit per pixel — is still often stored "as" an 8-bit image in practice, using only the values 0 and 255, for compatibility with standard image-processing software.

CONNECTION TO INFORMATION RETRIEVAL

- **Digital Libraries:** storage/bandwidth planning for large digitised collections depends directly on bit depth $\times$ resolution — a core practical concern in digitisation projects (recall Paper 07: Digitisation, file formats).

COMMON INTERVIEW / EXAM QUESTIONS

- How much raw storage does a 1000×1000 8-bit grayscale image require?

EXAMPLE

$1000 \times 1000 = 1,000,000$ pixels, each 1 byte (8 bits) $\Rightarrow 1,000,000$ bytes ≈ 976.6 KB (≈ 0.95 MB) of raw, uncompressed storage.

CHAPTER 5 — SUMMARY

A digital image is a rectangular grid of pixels; each pixel is addressed by (x,y) and holds an intensity $z = f(x,y)$. Because computers store data in bits, and 8 bits is the standard allotment per pixel, intensity is quantised into $2^8 = 256$ levels, labelled 0 to 255.

FURTHER READING

- Gonzalez & Woods, *Digital Image Processing* — Ch. 2 (Digital Image Fundamentals)
- Any introductory chapter on sampling and quantisation

[↑ Back to Table of Contents](#)

CHAPTER 06

Binary Images, Thresholds and Energy

Turning a landscape of 256 elevations into a simple map of "land" and "sea" — and measuring how much land there is.

6.1 The Binary Image

DEFINITION

A **binary image** is an image whose pixels can only take one of exactly two values — conventionally 0 (black) and 1 (or 255, white) — representing a hard decision of "background" versus "foreground/object" at every pixel.

WHY THIS CONCEPT IS NEEDED

Many tasks — separating text from paper, separating an object's silhouette from its background, preparing an image for skeleton extraction (Chapter 2) — don't need 256 shades of grey; they need one yes/no decision per pixel. Binary images strip away irrelevant shading detail and keep only the decision that matters: "is this pixel part of the object or not?"

REAL-LIFE INTUITION

Think of a stencil used for spray-painting: every point on the stencil is either "cut out" (paint passes through) or "solid" (paint is blocked) — there is no in-between shade. A binary image is the digital version of a stencil: a hard, two-valued mask of an image.

TECHNICAL EXPLANATION

A binary image is produced from a grayscale image by thresholding (Section 6.2): every pixel above the threshold becomes one value (e.g., 1/white), every pixel at or below it becomes the other (e.g., 0/black). In the topography analogy (Chapter 4), this is exactly "flooding" the terrain to a chosen sea level and recording only whether each point is submerged or dry.

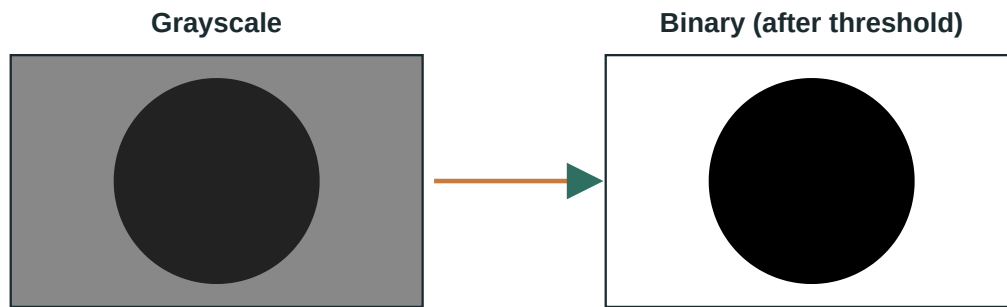


Fig 6.1 – Thresholding collapses many shades of grey into just two values: object (black circle) vs. background (white).

CONNECTION TO INFORMATION RETRIEVAL

- **OCR:** binarisation is almost always the very first step, isolating ink pixels from paper pixels before character segmentation.
- **Document Image Analysis:** binary masks are the direct input to skeletonisation and object extraction.

6.2 Threshold and $f^T(x,y)$

DEFINITION

A **threshold** T is a chosen cut-off intensity value. $f^T(x,y)$ denotes the *thresholded (binarised) image*: the result of applying that cut-off to every pixel of the original image $f(x,y)$.

WHY THIS CONCEPT IS NEEDED

We need a precise rule — not a vague one — for turning grayscale into binary. The threshold T is that rule's only parameter: pick one number, and every pixel's fate (object or background) is decided automatically by comparing it to T .

REAL-LIFE INTUITION

A threshold is like a passing mark on an exam: score above it, you "pass" (become foreground/white); score at or below it, you "fail" (become background/black). One single number decides the fate of every student (pixel), no matter how many students there are.

MATHEMATICAL EXPLANATION

$$f^T(x, y) = 1 \text{ if } f(x, y) > T, \quad \text{else } 0$$

$f(x, y)$ – the original grayscale intensity at (x, y) , a value 0–255.

T – the threshold: one chosen number, also between 0 and 255.

$f^T(x, y)$ – the output binary image: 1 (foreground/white) if the original pixel was brighter than T , and 0 (background/black) otherwise.

Why it exists: this is the simplest possible rule that turns a continuum of 256 values into a strict two-way decision, using only one number (T) that can be tuned per image.

WORKED EXAMPLE

EXAMPLE

Suppose $T = 128$. A pixel with $f(x, y) = 200 > 128$, so $f^T(x, y) = 1$ (foreground). A pixel with $f(x, y) = 90 \leq 128$, so $f^T(x, y) = 0$ (background). Changing T to 50 would reclassify that same 90-value pixel as foreground instead — this is why *choosing* the right T (Chapter 7.5) matters so much.

COMMON CONFUSION

Threshold is a single scalar number (like 128), not an image. $f^T(x, y)$ is the resulting binary *image* after that number has been applied everywhere.

CONNECTION TO INFORMATION RETRIEVAL

- **OCR / Document Image Analysis:** a badly chosen T can merge text into background (too high) or shatter it into speckled noise (too low) — directly affecting how well text can later be recognised and retrieved.

COMMON INTERVIEW / EXAM QUESTIONS

- Write the mathematical definition of image thresholding.
- What happens to a binary image's foreground area if the threshold T is increased?

Memory trick — "T for Tollgate": every pixel must pass through the tollgate at height T — clear it (brighter) and you're foreground, get stopped (darker or equal) and you're background.

6.3 Image Energy and $\Sigma f(x,y) = A$

DEFINITION

Image energy (in this simple, foundational sense) is the sum of all pixel intensities (or, for a binary image, the count of foreground pixels) across the whole image, written $\Sigma f(x,y) = A$. It is a single number that summarises "how much bright/foreground material" the whole image contains.

WHY THIS CONCEPT IS NEEDED

Sometimes we don't need to know the shape or location of anything in an image — we just need a single summary number, e.g., "how much ink is on this page," or "how much of this satellite tile is forest." Summing every pixel's value gives exactly that in one shot, and it is the simplest possible "global feature" of an image.

REAL-LIFE INTUITION

Imagine weighing an entire basket of fruit rather than weighing each fruit separately and remembering every individual number. The total weight is one useful summary

number, even though it throws away information about which fruit was heaviest. $\Sigma f(x,y)$ does the same thing for an image: it is the "total weight" of brightness.

TECHNICAL EXPLANATION

For a binary image, $\Sigma f(x,y)$ directly counts foreground pixels (since each contributes exactly 1), giving the *area* A of the object — this is precisely why the symbol A (area) is used. For a grayscale image, the same sum instead gives a rough measure of total "brightness energy," useful for quick global comparisons (e.g., is this scan overall darker or lighter than that one), though it is coarser than a histogram (Chapter 7), which preserves the full distribution rather than collapsing it to one number.

$$\Sigma f(x,y) = A = \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} f(x,y)$$

Σ (sigma) – the summation symbol: "add up everything that follows, over all listed values."

x, y – every column and row index in the image, ranging across the whole grid (0 to $N-1$, 0 to $M-1$).

$f(x,y)$ – the value being summed: the intensity (grayscale) or 0/1 flag (binary) at each pixel.

A – the resulting total: for a binary image, this equals the object's *area* in pixels; for a grayscale image, it is the image's total intensity "energy."

Why it exists: it is the most direct possible way to convert an entire 2-D grid of numbers into one single, comparable statistic – the starting point for any global image measurement.

WORKED EXAMPLE

EXAMPLE

A tiny 3×3 binary image (1 = foreground):

```
0 1 1
1 1 0
0 1 0
```

Summing all 9 values: $0+1+1+1+1+0+0+1+0 = 5$. So $A = 5$: the foreground object covers exactly 5 pixels' worth of area, out of the 9 pixels total.

Image type	What $\Sigma f(x,y)$ measures
Binary	Area (pixel count) of the foreground object
Grayscale	Total brightness "energy" of the whole image

CONNECTION TO INFORMATION RETRIEVAL

- **Image Retrieval:** total foreground area (A) is one of the simplest global shape/size features used to filter or rank candidate images before more expensive comparison.
- **Pattern Recognition:** object area is a basic, cheap-to-compute feature often used alongside shape descriptors like the skeleton (Chapter 2).

COMMON INTERVIEW / EXAM QUESTIONS

- What does $\Sigma f(x,y)$ represent for a binary image versus a grayscale image?

Memory trick — " Σ = Sum = Scale-to-one-number." Whenever you see Σ , think "the whole picture, boiled down to a single figure."

CHAPTER 6 – SUMMARY

Thresholding at a chosen value T converts a 256-level grayscale image $f(x,y)$ into a two-level binary image $f^T(x,y)$. Summing all pixel values, $\Sigma f(x,y) = A$, then gives a single global number: object area for binary images, or total brightness energy for grayscale images.

EXAM QUESTIONS (CHAPTER RECAP)

- Derive $f^T(x,y)$ from $f(x,y)$ and T .
- Compute $\Sigma f(x,y)$ for a given small binary matrix.

[↑ Back to Table of Contents](#)

CHAPTER 07

The Histogram — Turning Terrain into a Graph

The single most important tool in this course: a way of summarising an entire landscape of pixels in one simple picture.

7.1 What a Histogram Is

DEFINITION

An **image histogram** is a graph that shows, for every possible intensity value (0 to 255), how many pixels in the image have that exact value. The horizontal axis is intensity; the vertical axis is pixel count (or, equivalently, the percentage of pixels — Section 7.2).

WHY THIS CONCEPT IS NEEDED

A megapixel image contains millions of individual numbers — far too many to inspect one at a time. The histogram is a radical compression: it throws away *where* each pixel is located, and keeps only *how many* pixels have each brightness. This is exactly enough information to answer crucial questions like "is this image mostly dark or mostly bright?" and, crucially, "where is the best place to put a threshold?" (Section 7.5) — without ever needing to look at the image's actual layout.

REAL-LIFE INTUITION

Recall the geographic topography analogy (Chapter 4): if intensity is elevation, a histogram is exactly a geographer's chart of "how much land area exists at each elevation band" — regardless of *where* on the map that land is. Two mountain ranges with completely different shapes can have very similar histograms if they have similar amounts of land at each height; the histogram is a shape-blind, location-blind summary.

SIMPLE EXPLANATION

Walk through every pixel in the image, note its value (0–255), and add one tally mark to that value's bucket. After visiting every pixel, plot the tally counts as a bar chart, one bar per intensity value. That bar chart is the histogram.

TECHNICAL EXPLANATION

Formally, the histogram is a function $h(z)$ that counts how many pixels have intensity exactly equal to z . It is a 1-D summary derived from the full 2-D function $f(x,y)$ — deliberately discarding spatial information (the (x,y) part) to retain only the intensity distribution.

MATHEMATICAL EXPLANATION

$$h(z) = \text{number of pixels } (x,y) \text{ such that} \\ f(x,y) = z, \quad \text{for } z = 0, 1, \dots, 255$$

z — one specific intensity level being counted (e.g., $z = 128$).

$h(z)$ — the count of pixels that have exactly that intensity.

Why it exists: it converts the entire $M \times N$ grid of numbers into just 256 numbers (one count per possible intensity) — a massive, information-preserving-for-our-purposes compression, useful precisely because so many important image decisions (like thresholding) only need the distribution of values, not their spatial arrangement.

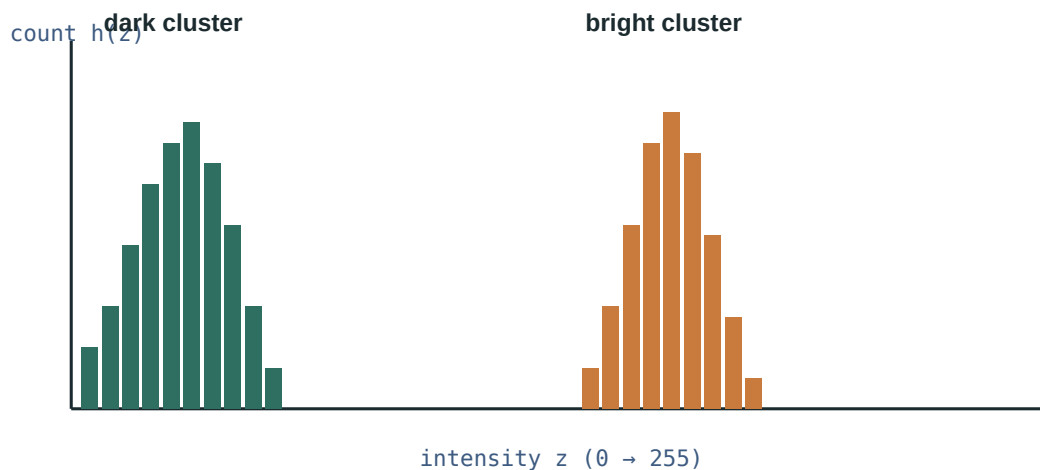


Fig 7.1 — A histogram with two humps: many dark pixels (ink/text, left) and many bright pixels (paper, right), with few pixels in between.

WORKED EXAMPLE

EXAMPLE

A tiny 4-pixel image has values 10, 10, 200, 200. Its histogram: $h(10) = 2$, $h(200) = 2$, and $h(z) = 0$ for every other z . Plotted, this shows two separate spikes — an extreme, miniature case of exactly the bimodal pattern (Section 7.3) seen in real scanned documents.

CONNECTION TO INFORMATION RETRIEVAL

- **Image Retrieval:** the histogram itself is one of the oldest and most widely used global features for comparing images — two images with similar histograms are often visually similar in tone.
- **OCR:** histogram shape is used to automatically decide a good threshold before character segmentation.
- **Multimedia Retrieval:** colour histograms (one per channel) are a classic baseline feature in image and video search systems.

7.2 Percentage of Pixels

DEFINITION

Rather than plotting the raw count $h(z)$, a **normalised histogram** plots the *percentage* (or fraction) of the image's total pixels that have intensity z .

WHY THIS CONCEPT IS NEEDED

Raw counts depend on image size — a 4K photo will have far larger bar heights than a thumbnail of the exact same scene, purely because it has more pixels overall. Converting to percentages removes this dependence, letting you fairly compare histograms of differently sized images (essential for image retrieval, where collection images are rarely all the same resolution).

MATHEMATICAL EXPLANATION

$$p(z) = h(z) / (M \times N) \times 100\%$$

$h(z)$ — the raw pixel count at intensity z (Section 7.1).

$M \times N$ — the total number of pixels in the image.

$p(z)$ — the percentage of the whole image's pixels that sit at intensity z .

Why it exists: dividing by the total makes the histogram a genuine *probability distribution* over intensities (all $p(z)$ sum to 100%), which is size-independent and directly comparable across images of different resolutions.

WORKED EXAMPLE

EXAMPLE

In a 10×10 image (100 pixels total), suppose 30 pixels have intensity 50. Then $p(50) = 30/100 \times 100\% = 30\%$. Regardless of whether this image were scaled up to 1000×1000 while keeping the same proportions, $p(50)$ would stay 30% — only the raw count $h(50)$ would change.

CONNECTION TO INFORMATION RETRIEVAL

- **Image Retrieval:** percentage-normalised histograms are what's actually compared/indexed across a collection, since raw counts would unfairly penalise (or favour) larger images.

Memory trick — "Percentage frees you from size": normalising by $M \times N$ is what lets you compare a phone photo and a scanned poster on equal footing.

7.3 Unimodal, Bimodal, Multimodal Histograms

DEFINITION

A histogram is **unimodal** if it has one dominant hump (peak), **bimodal** if it has two distinct humps, and **multimodal** if it has three or more distinct humps.

WHY THIS CONCEPT IS NEEDED

The number and shape of humps in a histogram tells us, at a glance, how many "distinct materials" or "distinct regions" the image likely contains — and, critically, whether simple thresholding (Chapter 6) is even an appropriate tool for this image (Section 7.5).

REAL-LIFE INTUITION

Back to the topography analogy: a unimodal histogram is like a landscape that is almost entirely one elevation (a flat plain) with a bit of variation — hard to split into "high" vs "low" meaningfully. A bimodal histogram is like a landscape with clearly separated lowlands and highlands (e.g., a valley and a plateau) — ideal for a clean threshold at the "valley floor" between them. A multimodal histogram is like a landscape with several distinct terraces at different heights — no single sea level cleanly separates "land" from "not-land."

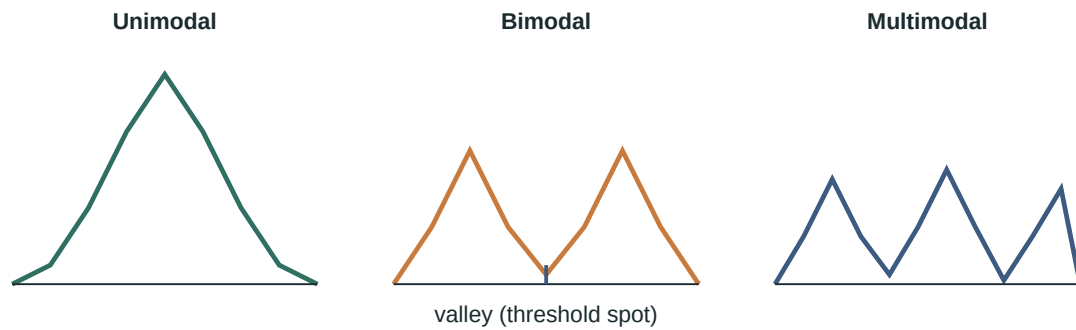


Fig 7.2 – Unimodal (one material), bimodal (two materials – e.g. ink vs paper), and multimodal (several materials) histogram shapes.

Shape	Typical cause	Thresholding implication
Unimodal	Nearly uniform scene (one dominant surface)	Hard to find a meaningful split point
Bimodal	Two clearly distinct regions (e.g. text vs. background)	Ideal — threshold at the valley between the two peaks
Multimodal	Several distinct regions/materials	Needs multiple thresholds or smarter, adaptive methods

WORKED EXAMPLE

EXAMPLE

A clean scan of typed black text on white paper produces a strongly bimodal histogram: one peak near 0–40 (ink) and another near 220–255 (paper), with very few pixels in between. This is precisely the "textbook" case where a single global threshold works beautifully.

CONNECTION TO INFORMATION RETRIEVAL

- **OCR:** a bimodal histogram is the green light for simple global thresholding; a multimodal one signals that the document (e.g. a photo embedded in text, or uneven lighting) needs more advanced, adaptive binarisation before OCR will work well.

COMMON INTERVIEW / EXAM QUESTIONS

- Why is a bimodal histogram considered ideal for simple global thresholding?
- Give a real-world scenario that would produce a multimodal histogram.

Memory trick — "uni-one, bi-two, multi-many": count the humps, name the mode.

7.4 Peaks and Valleys

DEFINITION

A **peak** in a histogram is a local maximum — an intensity value with more pixels than its immediate neighbours on both sides. A **valley** is a local minimum — an intensity value with fewer pixels than its immediate neighbours on both sides.

WHY THIS CONCEPT IS NEEDED

Peaks tell us where the "dominant materials" of the image sit on the brightness scale (e.g., ink, paper). Valleys tell us where *few* pixels exist — precisely the safest place to cut the image into foreground and background, since cutting there misclassifies the fewest pixels.

REAL-LIFE INTUITION

Back to the terrain: a histogram peak is a mountain summit — lots of land at that height. A histogram valley is a mountain pass — very little land at that height, because it's a transition zone between two summits. Choosing where to place a border fence, you'd naturally pick the pass (valley), not the middle of a summit (peak), since fewer people/pixels actually live there.

TECHNICAL EXPLANATION

Mathematically, z is a peak if $h(z) > h(z-1)$ and $h(z) > h(z+1)$; z is a valley if $h(z) < h(z-1)$ and $h(z) < h(z+1)$. In a clean bimodal histogram, there are exactly two peaks and one valley between them — and that valley is exactly where Section 7.5's threshold should be placed.

CONNECTION TO INFORMATION RETRIEVAL

- **Document Image Analysis:** automatic threshold-selection algorithms (e.g., simplified versions of Otsu's method) work by explicitly hunting for the histogram valley between two peaks.

Memory trick — "Peaks are popular, valleys are lonely": pixels crowd at peaks and avoid valleys — so valleys are the safest fence-lines.

7.5 Choosing a Threshold from the Histogram

DEFINITION

The standard rule for choosing a good threshold T (Chapter 6) from a bimodal histogram is to set **T = the intensity value at the valley between the two peaks.**

WHY THIS CONCEPT IS NEEDED

Chapter 6 introduced T as "just some chosen number" — but chosen how? The histogram answers this precisely: since the valley has the fewest pixels, placing T there misclassifies the smallest possible number of pixels, giving the cleanest possible split between foreground and background.

REAL-LIFE INTUITION

If you must build a fence dividing a valley village from a plateau village, you build it at the actual mountain pass (few people live there, so few people end up on the "wrong side"), not halfway up either village's own hillside (where you'd cut straight through a crowded neighbourhood).

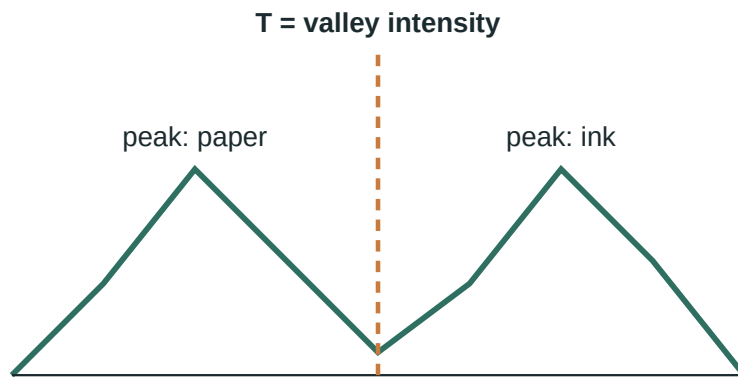


Fig 7.3 – The threshold is placed exactly at the valley, minimising misclassified pixels on both sides.

WORKED EXAMPLE

EXAMPLE

A histogram shows a peak at $z=30$ (ink), a valley at $z=110$ (very few pixels), and a peak at $z=210$ (paper). Setting $T = 110$ in $f^T(x,y) = 1$ if $f(x,y) > 110$ else 0 cleanly separates the two materials, since almost no real pixel sits exactly at 110 to be misclassified either way.

TIP FOR EXAMS

If asked to "choose a threshold given a histogram sketch," always look for the valley between the two tallest peaks — examiners usually draw an obviously bimodal shape specifically to test this rule.

CONNECTION TO INFORMATION RETRIEVAL

- **OCR:** automatic (as opposed to manually chosen) thresholding is what makes large-scale document digitisation for full-text search possible without a human tuning every single scan by hand.

COMMON INTERVIEW / EXAM QUESTIONS

- Why is the histogram valley the preferred location for a threshold, rather than the midpoint between the two peak intensities?

Memory trick — "Fence at the pass, not the peak."

Why the Histogram Is Useful — A Recap

Pulling Sections 7.1–7.5 together: the histogram is useful because it (1) compresses an entire image into a manageable 1-D summary, (2) reveals how many distinct materials/regions likely exist (unimodal/bimodal/multimodal), and (3) directly tells you where to place a threshold (the valley) if binarisation is the goal — turning a vague, manual decision into a precise, data-driven one.

CHAPTER 7 — SUMMARY

The histogram $h(z)$, often normalised into percentages $p(z)$, reveals the shape of an image's brightness distribution. Peaks mark dominant materials; valleys mark the best places to threshold. A bimodal histogram with a clear valley is the easiest case for clean binarisation.

EXAM QUESTIONS (CHAPTER RECAP)

- Define $h(z)$ and $p(z)$ and explain the difference.
- Sketch a bimodal histogram and mark where you would place a threshold, with justification.

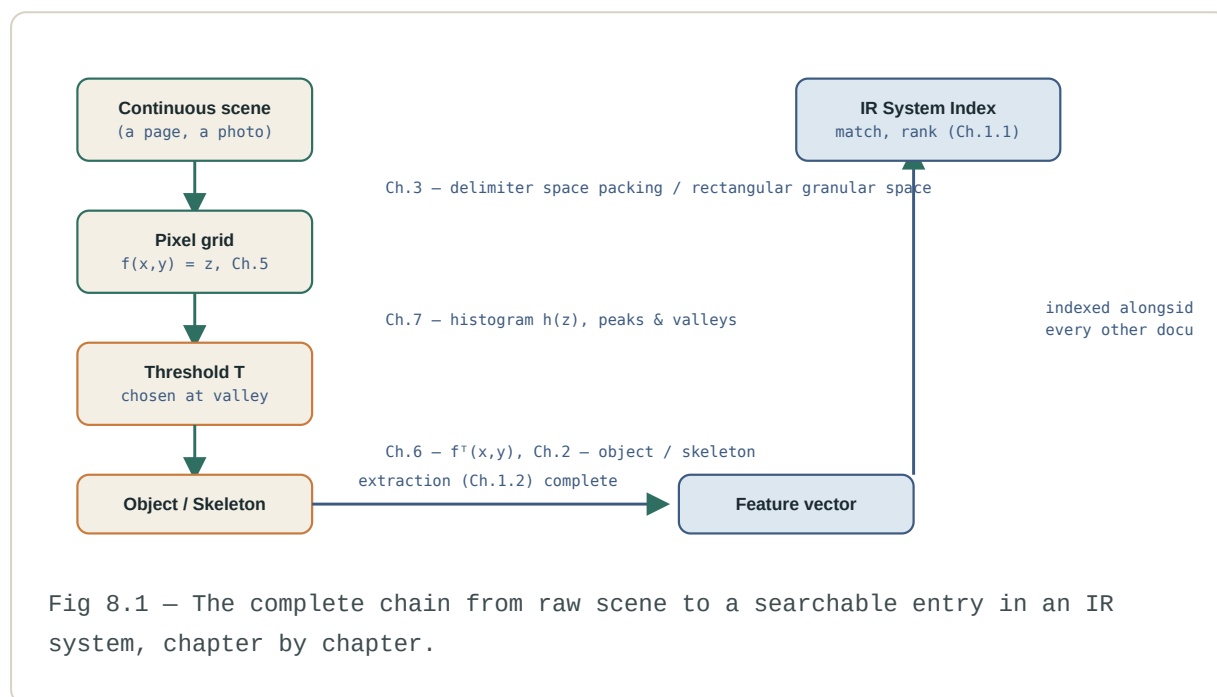
[↑ Back to Table of Contents](#)

CHAPTER 08

Synthesis — From Pixels to Search Engines

The full pipeline, laid out end to end, so you can see exactly how one lecture's worth of ideas becomes a working retrieval system.

Here is the complete chain, using this chapter's own vocabulary at every step:



THE STORY, IN WORDS

A continuous scene (a manuscript page, a photograph) is first **packed** into a rectangular grid of **pixels** (Chapters 3 & 5), each holding an intensity value governed by $f(x,y) = z$, quantised into 256 levels because of how binary storage works. The whole grid's brightness distribution is then summarised by a **histogram** (Chapter 7); its peaks reveal the dominant materials (ink vs. paper) and its valley suggests the best **threshold** (Chapter 6) for turning the grayscale image into a clean binary image $f^T(x,y)$. From that binary image, connected regions become **objects**, which can be thinned to **skeletons** (Chapter 2) — this entire chain is **extraction** (Chapter 1.2), producing a compact feature representation of one document. Only once this has been done for every document in a collection does true **Information Retrieval** begin: indexing those features and matching/ranking them against a user's query (Chapter 1.1).

WHY YOUR PROFESSOR TEACHES IT THIS WAY

Document Image Analysis, Mathematical Morphology, Pattern Recognition, Computer Vision and Information Retrieval are not five separate subjects to Dr. Sagar — they are five different vantage points on the exact same pipeline you've just traced. Mathematical morphology gives the tools for object/skeleton extraction and the topography analogy; pattern recognition classifies the resulting objects/skeletons; computer vision is the broader discipline these techniques belong to; and Information Retrieval is simply what happens once those extracted representations are indexed and searched, at scale, across a whole collection.

Course area	What it contributes to the pipeline
Document Image Analysis	The whole Chapter 3–6 pipeline, applied specifically to scanned pages
Mathematical Morphology	Rigorous tools for thresholding, object extraction, skeletonisation (Ch.2, Ch.6)
Pattern Recognition	Classifying the objects/skeletons once extracted
Computer Vision	The overarching field that includes all image-understanding tasks above
Information Retrieval	Indexing, matching and ranking the extracted representations across a collection

HOW TO REVISE THIS CHAPTER

If you can redraw Fig 8.1 from memory, labelling every arrow with the correct chapter number and formula, you understand the entire first lecture of this course — this diagram *is* the syllabus in miniature.

COMMON INTERVIEW / EXAM QUESTIONS

- Trace the full pipeline from a raw scanned page to a searchable digital library entry, naming every intermediate representation.
- Why does Information Retrieval depend on prior extraction steps when the "documents" are images rather than plain text?

- How do mathematical morphology and information retrieval relate, according to this pipeline?

Memory trick — "Pack, Picture, Plot, Pick, Pull, Ping": Pack the space, form the Picture (pixels), Plot the histogram, Pick a threshold, Pull out the object/skeleton, Ping the index (retrieve).

KEY POINTS TO REMEMBER

- Every concept in this lecture is a step in one continuous pipeline, not an isolated fact.
- Extraction (Chapters 2–7) happens per-document; Retrieval (Chapter 1.1) happens across the whole collection.
- The topography analogy (Chapter 4) is the mental thread tying mathematical morphology, GIS, and document image analysis together.

WHOLE-LECTURE SUMMARY

Space is packed into pixels; pixels form a function $f(x,y)=z$ quantised to 256 levels; the function's histogram reveals peaks and valleys; a valley gives a good threshold; thresholding yields a binary image whose objects and skeletons are extracted; extracted representations, once indexed across a collection, are what Information Retrieval actually searches.

FURTHER READING

- Gonzalez & Woods, *Digital Image Processing* — image fundamentals and thresholding chapters
- Chowdhury, G.G., *Introduction to Modern Information Retrieval* — Paper 11's core reference
- Any introduction to Mathematical Morphology (Serra, or lecture notes on morphological image analysis)

[↑ Back to Table of Contents](#)

Read in chapter order once, then use the Table of Contents as a revision index before exams.